บทที่ 11: การจัดการ state ขั้นสูงด้วย Provider และ Consumer

11.1 บทนำ: ต่อยอดองค์ความรู้เกี่ยวกับ Provider

ในบทที่ 10 เราได้ทำความเข้าใจกลไกพื้นฐานของแพ็กเกจ Provider และได้เรียนรู้วิธีการจัดการสถานะเบื้องต้นโดยใช้ ChangeNotifierProvider ร่วมกับ context.watch<T>() และ context.read<T>(). แนวทางดังกล่าวเป็นแนวทางเบื้องต้นที่มีประสิทธิภาพ ซึ่งช่วยให้เราสามารถแยกตรรกการจัดการสถานะออกจากส่วนติดต่อผู้ใช้ (UI) ได้สำเร็จ

บทที่ 10 ปิดท้ายไว้ด้วยคำถามที่ยังไม่ได้ตอบ นั่นคือ ถ้าแอปพลิเคชันจริงไม่ได้มี state เพียงตัวเดียวเหมือนแอปนับเลข แต่มีทั้งระบบคูปอง ระบบแนะนำสินค้า ระบบติดตามขนส่ง และประวัติการแชท การยัดตรรกะทั้งหมดไว้ใน provider ตัวเดียวจะทำให้มันบวมจนกลายเป็นคลาสขนาดหลายพันบรรทัดหรือไม่ และถ้าต้องหั่นออกเป็นส่วนย่อย แต่ละส่วนจะรับรู้การเปลี่ยนแปลงของกันและกันได้อย่างไร บทนี้คือคำตอบของคำถามนั้น

นอกจากนี้ บทที่ 10 ยังทิ้งท้ายไว้ว่าการใช้ context.watch<T>() ทั้งเมธอด build เป็นการรับสัญญาณที่กว้างเกินจำเป็น และได้เอ่ยถึงเครื่องมือสองตัวที่ช่วยตีกรอบให้แคบลง คือ Consumer<T> และ context.select<T, R>() โดยยกยอดรายละเอียดมาไว้ในบทนี้

อย่างไรก็ตาม เมื่อแอปพลิเคชันมีความซับซ้อนเพิ่มขึ้น เราจะเผชิญกับความท้าทายที่ต้องการเครื่องมือที่มีความสามารถเฉพาะทางมากขึ้น:

1. **ด้านประสิทธิภาพ (Performance):** ลองพิจารณาแอปพลิเคชันเครือข่ายสังคม (Social Media) ที่มีหน้าแสดงรายการโพสต์ (Feed) ซึ่งสามารถเลื่อนดูได้ต่อเนื่อง แต่ละโพสต์ประกอบด้วยรูปภาพ, ข้อความ, ชื่อผู้ใช้, ปุ่ม "ถูกใจ" และตัวนับจำนวนการถูกใจ หากเราใช้ context.watch<T>() ในวิดเจ็ตหลักของแต่ละโพสต์ ทุกครั้งที่มีการกดถูกใจในโพสต์ใดโพสต์หนึ่ง จะส่งผลให้ **วิดเจ็ตทั้งหมดของโพสต์นั้น** ถูกสร้างขึ้นใหม่ (rebuild) ทั้งๆ ที่ข้อมูลที่เปลี่ยนแปลงมีเพียงตัวเลขจำนวนการถูกใจเท่านั้น การสร้างวิดเจ็ตใหม่โดยไม่จำเป็นซ้ำๆ ในรายการที่มีข้อมูลจำนวนมาก อาจส่งผลให้การทำงานของแอปพลิเคชันหน่วงหรือไม่ลื่นไหล และสิ้นเปลืองทรัพยากรของระบบโดยใช่เหตุ
2. **ด้านการจัดระเบียบโครงสร้าง (Organization):** ลองพิจารณาแอปพลิเคชันพาณิชย์อิเล็กทรอนิกส์ (E-commerce) ซึ่งโดยทั่วไปจะต้องจัดการสถานะหลายส่วน เช่น UserProvider สำหรับข้อมูลผู้ใช้, ProductProvider สำหรับรายการสินค้า, และ CartProvider สำหรับตะกร้าสินค้า การนำ ChangeNotifierProvider มาซ้อนกันหลายชั้นในไฟล์ main.dart จะทำให้เกิดโครงสร้างที่ซ้อนกันลึก (deeply nested structure) ซึ่งลดทอนความสามารถในการอ่าน (readability) และเพิ่มความซับซ้อนในการบำรุงรักษาเมื่อต้องการเพิ่มหรือลบ Provider ในอนาคต

ในบทนี้ เราจะศึกษากลไกขั้นสูงของ Provider ที่ถูกออกแบบมาเพื่อรับมือกับความท้าทายเหล่านี้โดยเฉพาะ:

- **Consumer Widget:** เราจะเรียนรู้เทคนิคในการกำหนดขอบเขตการสร้างวิดเจ็ตใหม่ให้กระทบเฉพาะส่วนที่เล็กที่สุดที่ต้องการการอัปเดตข้อมูลจริงๆ (เช่น เฉพาะวิดเจ็ต Text ที่แสดงจำนวนการถูกใจ) เพื่อเพิ่มประสิทธิภาพการทำงานสูงสุดให้กับแอปพลิเคชัน
- **MultiProvider Widget:** เราจะเรียนรู้วิธีการจัดระเบียบ Provider หลายๆ ตัวให้อยู่ในรูปแบบของรายการที่อ่านง่าย ซึ่งเปรียบเสมือนการจัดหมวดหมู่เอกสาร ทำให้โครงสร้างโค้ดสะอาดและง่ายต่อการบำรุงรักษาในระยะยาว
- **context.select<T, R>() และ Selector:** เราจะเรียนรู้การเจาะจงลงไปถึงระดับ ค่าข้อมูลรายตัว ว่าวิดเจ็ตนี้สนใจเฉพาะฟิลด์ใดฟิลด์หนึ่งของ provider เท่านั้น ถ้าฟิลด์อื่นเปลี่ยนก็ไม่ต้องปลุกให้สร้างใหม่
- **การแบ่งความรับผิดชอบของ Provider:** เราจะเรียนรู้เกณฑ์การหั่น provider ขนาดใหญ่ออกเป็นหลายตัวตามขอบเขตความรับผิดชอบ และวิธีให้ provider ตัวหนึ่งพึ่งพาข้อมูลของอีกตัวหนึ่งผ่าน ChangeNotifierProxyProvider

เราจะนำองค์ความรู้เหล่านี้มาประยุกต์ใช้ในส่วนปฏิบัติการ โดยการสร้างแอปพลิเคชันสองหน้าจอขึ้นมาใหม่ทั้งหมด ซึ่งจะจำลองสถานการณ์ที่พบได้บ่อยในการพัฒนาแอปพลิเคชันจริง คือ การแสดงรายการข้อมูล, การแก้ไขข้อมูลในหน้าจอแยก, และการส่งข้อมูลที่อัปเดตแล้วกลับมายังหน้าจอหลัก ทั้งหมดนี้จะดำเนินการโดยใช้สถานะที่จัดการในหน่วยความจำ (In-memory) ซึ่งเป็นพื้นฐานที่สำคัญก่อนที่เราจะประยุกต์ใช้ฐานข้อมูลในบทถัดไป

11.2 การเพิ่มประสิทธิภาพด้วย Consumer Widget

context.watch<T>() เป็นคำสั่งที่สะดวก แต่เป็นการสั่งให้สร้างวิดเจ็ตใหม่ทั้งโครงสร้าง ในหลายสถานการณ์ เราต้องการเครื่องมือที่สามารถทำงานได้อย่างแม่นยำกว่านั้น ข้อจำกัดหลักของ context.watch<T>() คือการทำให้เมธอด build **ทั้งหมด** ของวิดเจ็ตที่เรียกใช้คำสั่งนี้ ต้องทำงานใหม่ทุกครั้งที่ Provider มีการเปลี่ยนแปลง

อุปมา: เปลี่ยนกระเบื้องบิ่นแผ่นเดียว ไม่ใช่ทุบห้องน้ำทั้งห้อง

ข้อจำกัดข้างต้นเห็นภาพได้ชัดถ้าเปรียบกับการซ่อมบ้าน สมมติว่ากระเบื้องห้องน้ำบิ่นไปเพียงแผ่นเดียว สิ่งที่เราต้องการคือเปลี่ยนกระเบื้องแผ่นนั้น แต่ผู้รับเหมากลับบอกว่าวิธีเดียวที่ทำได้คือทุบห้องน้ำทิ้งทั้งห้องแล้วสร้างใหม่

ผลลัพธ์สุดท้ายถูกต้อง เพราะเราได้กระเบื้องแผ่นใหม่จริง แต่ต้นทุนด้านเวลาและทรัพยากรนั้นสอบตกอย่างสิ้นเชิง และถ้าต้องทุบห้องน้ำทุกครั้งที่กระเบื้องบิ่นหนึ่งแผ่น นั่นคือฝันร้ายของการจัดการหน่วยความจำ นี่คือสิ่งที่เกิดขึ้นจริงเมื่อ context.watch<T>() ถูกวางไว้บนสุดของวิดเจ็ตขนาดใหญ่

สิ่งที่สถาปัตยกรรมที่ดีต้องการจึงเป็นเครื่องมือที่ทำงานเหมือนการผ่าตัดที่มีความแม่นยำสูง คือเข้าไปเปลี่ยนเฉพาะกระเบื้องแผ่นนั้นโดยไม่กระทบโครงสร้างส่วนอื่น และเครื่องมือตัวนั้นคือ Consumer

แนวคิดเบื้องหลังของ Consumer คือการพลิกทิศทางของการรับฟังข้อมูล แทนที่จะเอาหูไปแนบไว้ที่ยอดสุดของโครงสร้างแล้วฟังลงมาทั้งต้น (top-down) เราย้ายจุดรับฟังลงไปไว้เฉพาะกิ่งเล็ก ๆ ที่แสดงข้อมูลซึ่งเปลี่ยนแปลงบ่อยจริง ๆ เท่านั้น

Consumer คือเครื่องมือที่ช่วยให้สามารถกำหนดขอบเขตการสร้างใหม่ได้อย่างแม่นยำ มันทำหน้าที่ **สร้างเฉพาะส่วนของ widget tree ที่จำเป็นต้องอัปเดตตามสถานะจริงๆ.**

- **หลักการทำงาน:** แทนที่จะเรียก watch ที่ส่วนบนของเมธอด build เราจะนำ Consumer<YourProviderType> ไปห่อหุ้มเฉพาะวิดเจ็ตที่ต้องการข้อมูลจาก Provider เท่านั้น
- **พารามิเตอร์ builder:** Consumer รับฟังก์ชัน builder ซึ่งจะถูกเรียกใช้ใหม่ทุกครั้งที่มีการเรียก notifyListeners() ใน Provider ที่เกี่ยวข้อง [1] ฟังก์ชัน builder นี้จะได้รับ context, **อินสแตนซ์ของ Provider ที่ต้องการ**, และ child (เป็นพารามิเตอร์ทางเลือก สำหรับส่งผ่านวิดเจ็ตที่ไม่ต้องการให้ถูกสร้างใหม่) [2], [3]

เตรียมโปรเจกต์ตัวอย่างสำหรับลองรันตาม

ตัวอย่างในหัวข้อนี้อ้างถึง UserProvider ซ้ำหลายครั้ง แต่ยังไม่เคยแสดงโค้ดของ provider ตัวนั้นให้เห็น หัวข้อย่อยนี้จึงรวบรวมโค้ดตั้งต้นทั้งหมดไว้ เพื่อให้ผู้อ่านสร้างโปรเจกต์เล็ก ๆ ขึ้นมาทดลองรันตามได้จริง แล้วเห็นผลลัพธ์ใน Debug Console ด้วยตาตัวเอง ไม่ใช่อ่านโค้ดที่ยกมาลอย ๆ

ขั้นตอนมีเพียงห้าขั้น ใช้เวลาไม่เกินสิบนาที

- **ขั้นที่ 1** สร้างโปรเจกต์ใหม่ด้วยคำสั่ง flutter create provider_consumer_demo
- **ขั้นที่ 2** เปิด pubspec.yaml เพิ่ม provider ในส่วน dependencies แล้วสั่ง flutter pub get
- **ขั้นที่ 3** สร้างไฟล์ lib/providers/user_provider.dart ตามโค้ดชุดแรกด้านล่าง
- **ขั้นที่ 4** แทนที่เนื้อหาใน lib/main.dart ด้วยโค้ดชุดที่สอง แล้ววางคลาส UserProfileWidget และ UserProfileWidgetEfficient จากหัวข้อถัดไปต่อท้ายไฟล์เดียวกัน
- **ขั้นที่ 5** รันแอปแบบ Full Restart กดปุ่มมุมขวาล่างเพื่อเปลี่ยนชื่อเล่น แล้วสังเกตว่าข้อความจากคำสั่ง print ตัวใดปรากฏบ้างในแต่ละครั้งที่กด

```dart
// lib/providers/user_provider.dart
import 'package:flutter/foundation.dart';

class UserProvider extends ChangeNotifier {
  String _nickname = 'ต้นกล้า';
  final String _realName = 'ณัฐพงษ์ ใจดี';
  final String _profileImageUrl = 'https://picsum.photos/200';
  int _editCount = 0;

  String get nickname => _nickname;
  String get realName => _realName;
  String get profileImageUrl => _profileImageUrl;

  // เปลี่ยนเฉพาะ 'ชื่อเล่น' เพื่อใช้ทดสอบว่าวิดเจ็ตใดถูกสร้างใหม่บ้าง
  void changeNickname() {
    _editCount++;
    _nickname = 'ต้นกล้า คนที่ $_editCount';
    notifyListeners(); // ประกาศให้ผู้ที่รับฟังอยู่ทราบว่าข้อมูลเปลี่ยนแล้ว
  }
}
```

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/user_provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => UserProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('ทดลองขอบเขตการ rebuild')),
        // สลับไปใช้ UserProfileWidgetEfficient เพื่อเทียบผลใน Debug Console
        body: Center(child: UserProfileWidget()),
        floatingActionButton: FloatingActionButton(
          // ใช้ read เพราะปุ่มไม่ต้องเปลี่ยนหน้าตาตาม state ตามที่เรียนมาในบทที่ 10
```

```
          onPressed: () => context.read<UserProvider>().changeNickname(),
          child: const Icon(Icons.edit),
        ),
      ),
    );
  }
}
```

เมื่อโครงพร้อมแล้ว ให้เทียบโค้ดสองเวอร์ชันต่อไปนี้ โดยสลับบรรทัด body ใน main.dart ไปมาระหว่าง UserProfileWidget กับ UserProfileWidgetEfficient แล้วดูว่าข้อความใน Debug Console ต่างกันอย่างไรเมื่อกดปุ่มเปลี่ยนชื่อเล่นสามครั้งติดกัน

เปรียบเทียบการทำงานระหว่าง context.watch และ Consumer:

```
// --- กรณีใช้ context.watch (สร้างใหม่ทั้งโครงสร้าง) ---
// สมมติว่านี่คือวิดเจ็ตแสดงข้อมูลผู้ใช้
class UserProfileWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // การเรียก watch ที่นี่ ทำให้ UserProfileWidget ทั้งหมดถูกสร้างใหม่
    // ทุกครั้งที่ UserProvider มีการเปลี่ยนแปลง แม้จะเปลี่ยนแค่ชื่อเล่นก็ตาม
    final userProvider = context.watch<UserProvider>();
    print('UserProfileWidget build() ถูกเรียก!');

    return Column(
      children: [
        // วิดเจ็ต CircleAvatar นี้จะถูกสร้างใหม่โดยไม่จำเป็น
        CircleAvatar(radius: 50, backgroundImage:
NetworkImage(userProvider.profileImageUrl)),
        // วิดเจ็ต Text นี้ก็ถูกสร้างใหม่โดยไม่จำเป็น
        Text(userProvider.realName, style: TextStyle(fontSize: 24)),

        // --- ส่วนนี้คือส่วนเดียวที่ควรจะเปลี่ยนแปลงจริงๆ ---
        Text('ชื่อเล่น: ${userProvider.nickname}'),
      ],
    );
  }
}
```

```
// --- กรณีใช้ Consumer (สร้างใหม่เฉพาะส่วน) ---
class UserProfileWidgetEfficient extends StatelessWidget {
  @override
```

```
  Widget build(BuildContext context) {
    // เมธอด `build` ของวิดเจ็ตแม่จะทำงานเพียงครั้งแรก
    print('UserProfileWidgetEfficient build() ถูกเรียก!');

    return Column(
      children: [
        // ส่วนเหล่านี้จะไม่ถูกสร้างใหม่เมื่อ Provider เปลี่ยนแปลง
        // เราสามารถดึงข้อมูลครั้งเดียวด้วย `read` หากต้องการ
        CircleAvatar(radius: 50, backgroundImage:
NetworkImage(context.read<UserProvider>().profileImageUrl)),
        Text(context.read<UserProvider>().realName, style: TextStyle(fontSize:
24)),

        // ใช้ Consumer ห่อหุ้มเฉพาะวิดเจ็ต Text ที่แสดงชื่อเล่น
        Consumer<UserProvider>(
          builder: (context, userProvider, child) {
            // builder นี้เท่านั้นที่จะทำงานใหม่เมื่อชื่อเล่นเปลี่ยน
            print('Consumer สำหรับ "ชื่อเล่น" ถูกเรียก!');
            return Text('ชื่อเล่น: ${userProvider.nickname}');
          },
        ),
      ],
    );
  }
}
```

พารามิเตอร์ child ที่ตัวอย่างข้างต้นยังไม่ได้ใช้: สังเกตว่าฟังก์ชัน builder รับพารามิเตอร์สามตัวคือ context, อินสแตนซ์ของ provider และ child แต่ตัวอย่างข้างต้นใช้เพียงสองตัวแรก พารามิเตอร์ child มีไว้สำหรับกรณีที่ภายในขอบเขตของ Consumer ยังมีวิดเจ็ตบางส่วนที่ สร้างครั้งเดียวแล้วไม่ต้องเปลี่ยนตาม state อีกเลย

วิดเจ็ตที่ส่งเข้าทาง child จะถูกสร้างเพียงครั้งเดียวตอนที่ Consumer ถูกสร้างขึ้น จากนั้นทุกครั้งที่ builder ทำงานใหม่ Provider จะส่งอ็อบเจกต์เดิมตัวนั้นกลับเข้ามาให้ใช้ซ้ำ กลไกนี้มีประโยชน์มากเมื่อส่วนที่ไม่เปลี่ยนเป็นวิดเจ็ตที่สร้างยาก เช่น รูปภาพขนาดใหญ่ แผนภูมิ หรือรายการย่อยที่ซับซ้อน

```
// วิดเจ็ตสมมติที่สร้างยาก ใช้แทนส่วนที่ไม่ควรถูกสร้างใหม่ วางต่อท้าย main.dart ได้
class ExpensiveBadge extends StatelessWidget {
  const ExpensiveBadge({super.key});

  @override
  Widget build(BuildContext context) {
    print('ExpensiveBadge build() ถูกเรียก!');
    return const Icon(Icons.verified, size: 40);
  }
}

// การใช้พารามิเตอร์ child เพื่อกันไม่ให้ส่วนที่ 'แพง' ถูกสร้างใหม่
```

```
Consumer<UserProvider>(
  // สร้างครั้งเดียวตอน Consumer ถูกสร้าง แล้วส่งกลับเข้า builder ซ้ำทุกครั้ง
  child: const ExpensiveBadge(),
  builder: (context, userProvider, child) {
    print('builder ทำงาน แต่ ExpensiveBadge ไม่ถูกสร้างใหม่');
    return Row(
      children: [
        Text('ชื่อเล่น: ${userProvider.nickname}'),
        child!, // นำวิดเจ็ตเดิมกลับมาใช้ซ้ำ ไม่ต้องสร้างใหม่
      ],
    );
  },
)
```

ข้อควรระวังของตัวอย่างเปรียบเทียบข้างต้น: ในเวอร์ชันที่ใช้ Consumer เราดึง profileImageUrl และ realName ด้วย context.read<UserProvider>() ซึ่งอ่านค่า ณ ขณะนั้นเพียงครั้งเดียวและไม่รับฟังการเปลี่ยนแปลงต่อ วิธีนี้ถูกต้องก็ต่อเมื่อเรามั่นใจว่าสองค่านั้นจะไม่เปลี่ยนตลอดอายุของหน้าจอ หากภายหลังผู้ใช้สามารถเปลี่ยนรูปโปรไฟล์ได้ รูปบนหน้าจอจะค้างอยู่ที่รูปเดิมจนกว่าวิดเจ็ตแม่จะถูกสร้างใหม่ด้วยเหตุอื่น

หลักการจึงเป็นว่า ใช้ read กับค่าที่นิ่ง และใช้ Consumer หรือ context.select ห่อเฉพาะค่าที่เปลี่ยนได้ ถ้าไม่แน่ใจว่าค่าใดจะเปลี่ยนบ้าง ให้เลือกฝั่งที่ปลอดภัยไว้ก่อน คือรับฟังการเปลี่ยนแปลง แล้วค่อยปรับให้แคบลงเมื่อวัดผลแล้วพบว่าเป็นคอขวดจริง

ข้อดีเชิงปฏิบัติของ Consumer คืออะไร? ในแอปพลิเคชันจริงที่มี ListView ซึ่งแสดง UserProfileWidgetEfficient เป็นจำนวนมาก การป้องกันการสร้าง CircleAvatar และ Text อื่นๆ ที่ไม่จำเป็นในทุกรายการ จะช่วยให้การเลื่อนหน้าจอ (scrolling) มีความลื่นไหลมากขึ้น และลดการใช้ทรัพยากรของระบบ (CPU และ Memory) ลงได้อย่างมีนัยสำคัญ ส่งผลให้ผู้ใช้ได้รับประสบการณ์ที่ดีขึ้น

ข้อแนะนำ: ควรใช้ Consumer เมื่อต้องการเพิ่มประสิทธิภาพของแอปพลิเคชัน โดยการจำกัดขอบเขตการสร้างวิดเจ็ตใหม่ให้อยู่ในส่วนที่เล็กที่สุดเท่าที่จำเป็น

หลุมพราง: ถ้า Consumer ดีขนาดนี้ ทำไมไม่ครอบทุกวิดเจ็ตไปเลย

เมื่อเห็นประโยชน์ของ Consumer แล้ว คำถามที่ผู้เรียนมักถามต่อทันทีคือ ถ้าแอปของเรามีตัวแปรสถานะสิบตัว ทำไมไม่เอา Consumer ไปครอบทั้งสิบจุดแยกกันไปเลย จะได้รับประกันว่าทุกจุดอัปเดตเฉพาะส่วนของตัวเองร้อยเปอร์เซ็นต์

คำถามนี้ฟังดูมีเหตุผล แต่เป็นหลุมพรางที่นักพัฒนาตกลงไปบ่อยที่สุดข้อหนึ่ง เหตุผลอยู่ที่ข้อเท็จจริงง่าย ๆ ข้อเดียว คือ Consumer เองก็เป็นวิดเจ็ตชนิดหนึ่งเหมือนกัน มันจึงมีต้นทุนของตัวมันเอง

- **โครงสร้างลึกขึ้นโดยไม่จำเป็น** การนำ Consumer ไปครอบวิดเจ็ตย่อยทุกตัว เท่ากับเราสร้าง widget tree ที่ลึกและซับซ้อนขึ้นอีกหนึ่งชั้นต่อหนึ่งจุด ซึ่งก็คือ boilerplate ที่ต้องมาไล่อ่านและบำรุงรักษาภายหลัง
- **ภาระในการจัดสรรหน่วยความจำ** ฟังก์ชัน builder จำนวนมากที่ทำงานพร้อมกันสร้าง overhead ให้ระบบ ถ้าเราแบ่งย่อยเกินไป ค่าบริหารจัดการส่วนนี้อาจแพงกว่าทรัพยากรที่ประหยัดได้เสียอีก

กุญแจสำคัญจึงอยู่ที่ความพอดี ให้ใช้ Consumer ครอบเฉพาะกลุ่มข้อมูลที่เปลี่ยนแปลงบ่อยและมีผลต่อประสิทธิภาพจริง ๆ เท่านั้น ส่วนวิธีตรวจสอบว่าจุดใดเข้าข่ายนี้บ้าง จะกล่าวถึงในหัวข้อ 11.5

11.3 การตีกรอบให้แคบลงอีกขั้นด้วย context.select และ Selector

Consumer ช่วยตีกรอบ พื้นที่ ของการสร้างใหม่ให้เหลือเฉพาะวิดเจ็ตที่ห่อไว้ แต่ยังมีข้อจำกัดอยู่หนึ่งข้อ คือ builder ของ Consumer จะทำงานใหม่ทุกครั้งที่ provider เรียก notifyListeners() ไม่ว่าข้อมูลที่เปลี่ยนไปจะเป็นฟิลด์ที่วิดเจ็ตนั้นสนใจหรือไม่

ลองพิจารณา UserProvider ที่เก็บทั้ง nickname, realName และ profileImageUrl หากผู้ใช้แก้ไขรูปโปรไฟล์ provider จะประกาศการเปลี่ยนแปลงออกไป และ Consumer ที่ห่อข้อความแสดงชื่อเล่นไว้ก็จะทำงานใหม่ด้วย ทั้งที่ชื่อเล่นไม่ได้เปลี่ยนแม้แต่ตัวอักษรเดียว

context.select<T, R>() คือเครื่องมือที่แก้ปัญหานี้โดยตรง มันเปลี่ยนคำถามจาก "provider ตัวนี้เปลี่ยนหรือยัง" ไปเป็น "ค่าที่ฉันสนใจในตัวมันเปลี่ยนหรือยัง" เราส่งฟังก์ชันเข้าไปเพื่อบอกว่าต้องการค่าใด แล้ว Provider จะเปรียบเทียบค่าที่ได้กับค่าครั้งก่อนด้วยตัวดำเนินการ == ถ้าเท่ากันก็จะไม่สั่ง rebuild [4]

```dart
// rebuild เฉพาะตอน 'ชื่อเล่น' เปลี่ยน ฟิลด์อื่นของ UserProvider เปลี่ยนก็ไม่สนใจ
class NicknameLabel extends StatelessWidget {
  const NicknameLabel({super.key});

  @override
  Widget build(BuildContext context) {
    final nickname = context.select<UserProvider, String>(
      (provider) => provider.nickname,
    );
    return Text('ชื่อเล่น: $nickname');
  }
}
```

คลาสนี้นำไปวางในโปรเจกต์ provider_consumer_demo ที่เตรียมไว้ในหัวข้อ 11.2 ได้ทันที เพียงเปลี่ยนบรรทัด body ใน main.dart เป็น Center(child: NicknameLabel()) แล้วลองเพิ่มเมธอดที่แก้ไขฟิลด์อื่นใน UserProvider เพื่อยืนยันว่าข้อความนี้ไม่ถูกสร้างใหม่เมื่อฟิลด์อื่นเปลี่ยน

รูปแบบวิดเจ็ต: Selector<T, R> สำหรับกรณีที่ต้องการตีกรอบเฉพาะบางส่วนของเมธอด build เหมือนที่ Consumer ทำ แพ็กเกจ provider มีวิดเจ็ตชื่อ Selector ให้ใช้ ซึ่งเทียบได้กับการนำ Consumer มารวมกับ select โดยรับทั้ง selector, builder และ child [5]

```dart
// Selector = Consumer ที่กรองข้อมูลก่อน ใช้เมื่อไม่ต้องการ rebuild ทั้งเมธอด build
Selector<UserProvider, String>(
  selector: (context, provider) => provider.nickname,
  builder: (context, nickname, child) => Text('ชื่อเล่น: $nickname'),
)
```

กับดักที่พบบ่อยที่สุดของ select: ฟังก์ชันที่เราส่งเข้าไปต้องคืนค่าที่เปรียบเทียบด้วย == ได้อย่างมีความหมาย ค่าพื้นฐานอย่าง int, double, String และ bool ใช้ได้ทันที แต่ถ้าเราคืนค่าเป็น List, Map หรืออ็อบเจกต์ที่ไม่ได้นิยามตัวดำเนินการ == ไว้ ทุกครั้งที่เรียกจะได้อ็อบเจกต์คนละตัวกันเสมอ ผลคือ == ไม่เท่ากันตลอด และวิดเจ็ตก็จะถูกสร้างใหม่ทุกครั้ง ซึ่งแย่กว่าการใช้ Consumer ธรรมดาเสียอีก เพราะเสียเวลาไปกับการเปรียบเทียบเพิ่มโดยไม่ได้ประโยชน์

ตัวอย่างด้านล่างอ้างถึง ItemProvider ซึ่งเป็น provider ที่เก็บลิสต์ของสินค้า เราจะสร้างมันขึ้นมาจริงในปฏิบัติการท้ายบท ตอนนี้ขอให้มองเป็นเพียงภาพประกอบของกับดักที่กำลังพูดถึง

```dart
// ผิด: map().toList() คืน List ใหม่ทุกครั้ง การเทียบด้วย == จึงไม่มีวันเท่ากัน
final names = context.select<ItemProvider, List<String>>(
  (provider) => provider.items.map((item) => item.name).toList(),
);

// ถูก: เลือกค่าที่เปรียบเทียบด้วย == ได้ตรงไปตรงมา
final itemCount = context.select<ItemProvider, int>(
  (provider) => provider.items.length,
);
```

ข้อควรจำอีกสองข้อ ข้อแรก ฟังก์ชัน selector จะถูกเรียกทุกครั้งที่ provider ประกาศการเปลี่ยนแปลง จึงต้องเขียนให้เบาและไม่มีผลข้างเคียง ห้ามใส่การคำนวณหนักหรือการเรียก API ลงไป ข้อที่สอง context.select ต้องถูกเรียกภายในเมธอด build เท่านั้น ห้ามเรียกใน onPressed หรือใน callback อื่น เพราะมันคือการสมัครรับการเปลี่ยนแปลง ไม่ใช่การอ่านค่าครั้งเดียว ถ้าต้องการอ่านค่าใน callback ให้ใช้ context.read<T>() ตามที่เรียนมาในบทที่ 10

11.4 บันไดของการรับฟัง: เลือกเครื่องมือให้พอดีกับงาน

เมื่อรวมสิ่งที่เรียนมาในบทที่ 10 เข้ากับสองหัวข้อข้างต้น เราจะได้เครื่องมือสี่ระดับที่เรียงจากกว้างที่สุดไปแคบที่สุด การเลือกให้ถูกระดับคือหัวใจของการทำให้แอปพลิเคชันลื่นไหล

- **context.read<T>() — ไม่รับฟังเลย** ใช้เมื่อวิดเจ็ตต้องการเพียงสั่งงาน provider เช่นใน onPressed หน้าตาของวิดเจ็ตไม่ต้องเปลี่ยนตามผลลัพธ์
- **context.watch<T>() — รับฟังทั้งเมธอด build** ใช้เมื่อเกือบทุกส่วนของวิดเจ็ตนั้นต้องเปลี่ยนตาม state อยู่แล้ว หรือเมื่อวิดเจ็ตมีขนาดเล็กจนการตีกรอบเพิ่มไม่คุ้มกับความซับซ้อนของโค้ด
- **Consumer<T> — รับฟังเฉพาะพื้นที่ที่ห่อไว้** ใช้เมื่อวิดเจ็ตแม่มีส่วนประกอบหลายชิ้น แต่มีเพียงบางชิ้นที่ต้องเปลี่ยนตาม state
- **context.select<T, R>() และ Selector<T, R> — รับฟังเฉพาะค่าที่ระบุ** ใช้เมื่อ provider ตัวนั้นถือข้อมูลหลายฟิลด์ และมีการประกาศการเปลี่ยนแปลงบ่อยจากฟิลด์ที่วิดเจ็ตนี้ไม่ได้สนใจ

ลำดับการตัดสินใจที่แนะนำ ให้เริ่มจากสิ่งที่อ่านง่ายที่สุดก่อนเสมอ คือใช้ watch หรือ Consumer ให้แอปทำงานถูกต้องก่อน แล้วจึงวัดผลตามวิธีในหัวข้อถัดไป ถ้าพบว่ามีวิดเจ็ตที่ถูกสร้างใหม่ซ้ำ ๆ โดยไม่จำเป็นจนกระทบความลื่นไหล ค่อยไต่ลงมาใช้เครื่องมือที่แคบกว่าเฉพาะจุดนั้น

เหตุผลคือทุกขั้นที่แคบลงมีต้นทุนเป็นความซับซ้อนของโค้ดที่เพิ่มขึ้น การใส่ Selector ไว้ทุกจุดในแอปตั้งแต่ยังไม่มีปัญหาเรื่องประสิทธิภาพ คือการจ่ายด้วยความอ่านยากเพื่อแลกกับสิ่งที่วัดไม่ได้ ซึ่งขัดกับหลักการทำงานที่ดีพอ ๆ กับการปล่อยให้แอปกระตุก

11.5 พิสูจน์ด้วยการวัด: ดูให้เห็นว่าวิดเจ็ตใดถูกสร้างใหม่บ้าง

ข้อความว่า "Consumer ช่วยลดการ rebuild" จะยังเป็นเพียงคำกล่าวอ้าง จนกว่าเราจะมองเห็นด้วยตาตัวเองว่าวิดเจ็ตใดถูกสร้างใหม่บ้างในแต่ละครั้งที่ state เปลี่ยน หัวข้อนี้รวบรวมวิธีตรวจสอบสามระดับ ตั้งแต่ง่ายที่สุดไปจนถึงละเอียดที่สุด

- **ระดับที่ 1 วาง print ไว้ในเมธอด build และใน builder** เป็นวิธีที่เร็วและตรงไปตรงมาที่สุด ดังที่โค้ดตัวอย่างในหัวข้อ 11.2 ทำไว้ ให้กดปุ่มที่เปลี่ยน state แล้วสังเกตว่าข้อความใดปรากฏใน console บ้าง ถ้าตีกรอบได้ผลจริง ข้อความจากวิดเจ็ตแม่ต้องปรากฏเพียงครั้งเดียว ขณะที่ข้อความจาก builder ปรากฏทุกครั้งที่กด
- **ระดับที่ 2 เปิด debugPrintRebuildDirtyWidgets** Flutter จะพิมพ์ชื่อวิดเจ็ตทุกตัวที่ถูกสร้างใหม่ในแต่ละเฟรมให้อัตโนมัติ [6] เหมาะกับตอนที่ยังไม่รู้ว่าควรไปวาง print ตรงไหน

- **ระดับที่ 3 ใช้เครื่องมือวัดของ IDE และ Flutter DevTools ใน Android Studio หรือ IntelliJ ให้เปิด View > Tool Windows > Flutter Performance แล้วกด Show widget rebuild information จะเห็นจำนวนครั้งที่วิดเจ็ตแต่ละตัวถูกสร้างใหม่ในเฟรมล่าสุดและนับรวมตั้งแต่เข้าหน้าจอ พร้อมไอคอนหมุนในหน้าโค้ดตรงบรรทัดที่เกิดการ rebuild (ใช้ได้เฉพาะ debug build) [7] ส่วนใน Flutter DevTools หน้า Performance ให้เปิดตัวเลือก Track Widget Builds ในเมนู Enhance tracing จะเห็นเหตุการณ์ build() พร้อมชื่อวิดเจ็ตบน timeline ของแต่ละเฟรม ทั้งสองเครื่องมือละเอียดที่สุดและใช้หาคอขวดในแอปขนาดใหญ่ได้จริง [8]**

```dart
// lib/main.dart — เปิดเฉพาะตอนตรวจสอบ แล้วปิดกลับเมื่อเสร็จ
// ตัวแปรนี้มาพร้อม package:flutter/material.dart อยู่แล้ว ไม่ต้อง import เพิ่ม
void main() {
  debugPrintRebuildDirtyWidgets = true; // พิมพ์ชื่อวิดเจ็ตที่ถูกสร้างใหม่ทุกเฟรม
  runApp(const MyApp());
}
```

ข้อควรระวังเรื่องการตีความตัวเลข การวัดประสิทธิภาพต้องทำในโหมด profile ไม่ใช่โหมด debug เพราะโค้ดในโหมด debug ไม่ได้ผ่านการปรับให้เหมาะสม และมีการตรวจสอบเพิ่มเติมจำนวนมากที่ทำให้ช้ากว่าความเป็นจริงหลายเท่า [9] ส่วนคำสั่ง print เองก็มีต้นทุน หากวางไว้ในจุดที่ทำงานทุกเฟรมก็จะกลายเป็นตัวถ่วงเสียเอง

ตัวเลขอ้างอิงที่ควรจำคือ หน้าจอส่วนใหญ่ทำงานที่ 60 เฟรมต่อวินาที เท่ากับว่าแต่ละเฟรมมีเวลาราว 16 มิลลิวินาทีสำหรับงานทั้งหมด ตั้งแต่การ rebuild ไปจนถึงการวาดจริง ถ้ามีเฟรมใดใช้เวลาเกินกว่านั้น ผู้ใช้จะรู้สึกได้ทันทีว่าภาพสะดุด

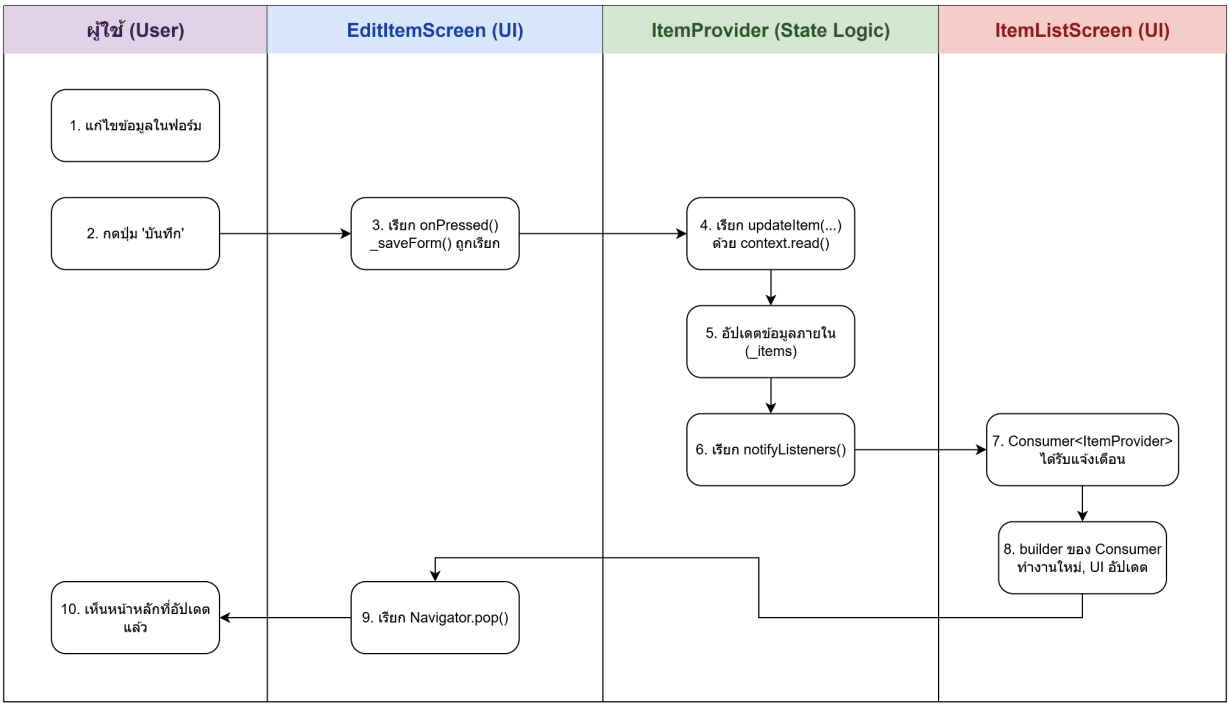
11.6 ภาพรวมการทำงาน: แผนภาพกระบวนการ (Swimlane Diagram)

เพื่อให้เห็นภาพรวมการทำงานของ Provider และ Consumer ในแอปพลิเคชันตัวอย่างของเรา (ปฏิบัติการบทที่ 11) ได้ชัดเจนยิ่งขึ้น ลองพิจารณาแผนภาพกระบวนการต่อไปนี้ ซึ่งแสดงลำดับการทำงานเมื่อผู้ใช้แก้ไขข้อมูล

แผนภาพนี้แบ่งการทำงานออกเป็น 4 ส่วน (หรือ "เลน") คือ:

- **ผู้ใช้ (User):** ผู้กระทำการในระบบ
- **EditItemScreen (UI):** หน้าจอสำหรับแก้ไขข้อมูล
- **ItemProvider (State Logic):** ส่วนจัดการสถานะและตรรกะทางธุรกิจ
- **ItemListScreen (UI):** หน้าจอหลักที่แสดงรายการและ "ฟัง" การเปลี่ยนแปลง

กระบวนการอัปเดตสถานะด้วย Provider



จากแผนภาพจะเห็นการ **"แยกหน้าที่"** อย่างชัดเจน:

- EditItemScreen ทำหน้าที่ **ส่งคำสั่ง** (context.read) เพื่อเปลี่ยนแปลงสถานะ แต่ **ไม่สนใจ** ว่าจะเกิดอะไรขึ้นต่อไป (fire and forget).
- ItemProvider ทำหน้าที่ **จัดการตรรกะ** และ **ประกาศการเปลี่ยนแปลง** (notifyListeners).
- ItemListScreen ผ่าน Consumer ทำหน้าที่ **รับฟังการเปลี่ยนแปลง** และ **อัปเดต UI** ตามสถานะใหม่.

นี่คือหัวใจของการจัดการสถานะที่ดี ซึ่งช่วยให้โค้ดแต่ละส่วนทำงานตามหน้าที่ของตนเอง ทำให้ง่ายต่อการทำความเข้าใจและบำรุงรักษา.

อุปมา: ลูกค้า พ่อครัว และพนักงานเสิร์ฟ

ลำดับการทำงานทั้งสามเลนข้างต้นเทียบได้กับระบบรับออเดอร์ในร้านอาหาร ซึ่งเป็นระบบที่ทุกคนคุ้นเคยอยู่แล้ว

- **หน้าจอแก้ไขคือลูกค้า** ลูกค้าเขียนออเดอร์ยื่นให้พนักงาน แล้วกลับไปนั่งรอที่โต๊ะ ไม่ต้องเดินไปยืนคุมหน้าเตาเอง นี่คือความหมายของ fire and forget
- **Provider คือพ่อครัว** รับออเดอร์มาปรุงอยู่หลังร้าน พอผัดกะเพราเสร็จก็สั่นกระดิ่ง ซึ่งก็คือการเรียก notifyListeners()
- **Consumer คือพนักงานเสิร์ฟ** ได้ยินเสียงกระดิ่งเมื่อไรก็รีบไปรับจานมาเสิร์ฟที่โต๊ะทันที โดยไม่ต้องคอยเดินไปถามพ่อครัวเป็นระยะว่าเสร็จหรือยัง

ประโยชน์ที่ได้มากกว่าความสวยงามของโครงสร้าง คือการทดสอบและการไล่หาข้อผิดพลาดที่ง่ายขึ้นอย่างเห็นได้ชัด เมื่อแต่ละฝ่ายมีหน้าที่เดียวและไม่ก้าวก่ายกัน อาการเสียแต่ละแบบจะชี้ไปที่ผู้ต้องสงสัยเพียงรายเดียว

ถ้าผู้ใช้กดบันทึกแล้วหน้าจอหลักไม่อัปเดต เราไปตรวจที่พ่อครัวว่าลืมสั่นกระดิ่งหรือเปล่า นั่นคือตรวจว่า provider เรียก notifyListeners() แล้วหรือยัง โดยไม่ต้องไปรื้อโค้ดฝั่งลูกค้าเลย และถ้าผู้ใช้พิมพ์ข้อมูลในฟอร์มไม่ได้ตั้งแต่แรก เราก็ไปดูที่หน้าจอแก้ไขจุดเดียว ไม่ต้องสงสัย provider

11.7 การจัดการสถานะหลายประเภทด้วย MultiProvider

เมื่อโปรเจกต์มีความซับซ้อน การมี Provider เพียงประเภทเดียวมักจะไม่เพียงพอ คุณอาจต้องการ AuthProvider สำหรับจัดการสถานะการเข้าสู่ระบบ และ CartProvider สำหรับจัดการสินค้าในตะกร้า การนำ Provider เหล่านี้มาซ้อนกันจะทำให้โครงสร้างโค้ดอ่านและทำความเข้าใจได้ยาก:

โครงสร้างที่ซ้อนกันลึกแบบนี้มีชื่อเรียกติดปากในหมู่นักพัฒนาว่า Pyramid of Doom หรือพีระมิดมรณะ อาการของมันคือโค้ดจะเยื้องลึกลงไปทางขวาเรื่อย ๆ ทุกครั้งที่เพิ่ม provider เข้ามาอีกหนึ่งตัว เหมือนการเอากล่องมาซ้อนในกล่อง ซ้อนเข้าไปอีกห้าหกชั้น

สิ่งที่ต้องเน้นคือ โครงสร้างแบบนี้ไม่ได้ทำให้แอปพลิเคชันทำงานผิดพลาด มันรันได้ตามปกติทุกประการ ปัญหาที่แท้จริงคือมันทำลายความสามารถในการอ่านโค้ดจนหมดสิ้น เมื่อเพื่อนร่วมทีมมารับช่วงต่อ หรือเมื่อเกิดบั๊กขึ้นมา การไล่หาว่าวงเล็บคู่นี้เปิดที่ provider ตัวไหนและปิดที่ตัวไหนกลายเป็นงานที่กินเวลาอย่างไม่สมเหตุสมผล

ยิ่งไปกว่านั้น เมื่อสมาชิกหลายคนแก้ไฟล์ main.dart พร้อมกันแล้วนำโค้ดมารวมกันในระบบ version control โอกาสที่วงเล็บจะตีกันจนโปรแกรมพังนั้นสูงมาก เพราะการเพิ่ม provider หนึ่งตัวกระทบการเยื้องของทุกบรรทัดที่อยู่ข้างใน

```
// โครงสร้างที่ซ้อนกัน (ไม่แนะนำ)
ChangeNotifierProvider(
  create: (context) => AuthProvider(),
  child: ChangeNotifierProvider(
    create: (context) => CartProvider(),
    child: ChangeNotifierProvider(
      create: (context) => ProductProvider(),
      child: const MyApp(),
    ),
  ),
)
```

MultiProvider คือเครื่องมือที่ช่วยให้คุณจัดระเบียบ Provider เหล่านี้ให้อยู่ในรูปแบบรายการที่เรียบง่ายและสะอาดตา เปรียบเสมือนเป็นศูนย์กลางในการประกาศ Provider ทั้งหมดของแอปพลิเคชัน

```
// ในไฟล์ main.dart (เป็นแนวทางที่แนะนำ)
import 'package:provider/provider.dart';
```

```dart
void main() {
  runApp(
    MultiProvider(
      providers: [
        // ประกาศ Provider ทั้งหมดที่ต้องการในรูปแบบรายการ
        ChangeNotifierProvider(create: (context) => AuthProvider()),
        ChangeNotifierProvider(create: (context) => CartProvider()),
        ChangeNotifierProvider(create: (context) => ProductProvider()),
      ],
      child: const MyApp(),
    ),
  );
}
```

ข้อดีของการใช้ MultiProvider คืออะไร? การใช้ MultiProvider ไม่ได้เปลี่ยนแปลงวิธีการทำงานของ Provider แต่เป็นการปรับปรุงโครงสร้างโค้ดของคุณโดยตรง:

- **ความสามารถในการอ่าน (Readability):** ทำให้สามารถตรวจสอบได้อย่างรวดเร็วว่ามี Provider ประเภทใดบ้างที่ถูกใช้งานในโปรเจกต์
- **ความสะดวกในการบำรุงรักษา (Maintainability):** การเพิ่ม, ลบ, หรือสลับลำดับ Provider สามารถทำได้โดยง่ายเพียงแค่แก้ไขรายการ providers
- **ลดความซับซ้อนของโครงสร้าง (Reduced Nesting):** ช่วยหลีกเลี่ยงการซ้อนวิดเจ็ตที่ลึกเกินความจำเป็น ทำให้โครงสร้างโดยรวมของแอปพลิเคชันเข้าใจง่ายขึ้น

การวางโครงสร้างที่ดีตั้งแต่เริ่มต้นด้วย MultiProvider จะช่วยลดความซับซ้อนและเพิ่มประสิทธิภาพในการพัฒนาระยะยาว โดยเฉพาะอย่างยิ่งในการทำงานร่วมกันเป็นทีม

คำถามที่ควรถามคือ ถ้าไม่ได้เร็วขึ้น แล้วมูลค่าที่แท้จริงอยู่ตรงไหน คำตอบคือ MultiProvider ไม่ได้เข้าไปเปลี่ยนกระบวนการทำงานหรือประสิทธิภาพของ provider แต่ละตัวแม้แต่น้อย การดึงข้อมูลและการ rebuild ยังทำงานด้วยตรรกะเดิมทุกประการ สิ่งที่มันให้คือการบำรุงรักษาในระยะยาวล้วน ๆ

เหตุผลที่คุณค่าข้อนี้สำคัญกว่าที่คิด อยู่ที่ข้อเท็จจริงของการทำงานจริง คือโค้ดถูกเขียนขึ้นเพียงครั้งเดียว แต่ถูกอ่านและถูกแก้ไขซ้ำแล้วซ้ำเล่าเป็นร้อยครั้ง การทำให้เพิ่ม provider ใหม่ได้ง่าย ลบตัวเก่าทิ้งได้โดยไม่กระทบบรรทัดอื่น จึงเป็นการลงทุนที่คืนทุนทุกครั้งที่มีคนเปิดไฟล์นี้

สิ่งที่ MultiProvider ไม่ได้เปลี่ยน: เบื้องหลังแล้ว MultiProvider ยังคงนำ provider แต่ละตัวมาซ้อนกันเป็นชั้น ๆ ให้เราเหมือนเดิม สิ่งที่เปลี่ยนคือรูปแบบการเขียนที่อ่านง่ายขึ้นเท่านั้น [10] ข้อเท็จจริงนี้มีผลสำคัญสองข้อที่ต้องรู้

- **ลำดับในรายการมีความหมาย** provider ที่ประกาศไว้ก่อนจะอยู่ชั้นบนของ widget tree เสมอ ดังนั้น provider ตัวที่ต้องอ่านข้อมูลของอีกตัวหนึ่ง ต้องถูกประกาศไว้ หลัง ตัวที่มันพึ่งพา มิฉะนั้นจะหาไม่เจอและเกิดข้อผิดพลาดขณะทำงาน
- **provider ถูกสร้างแบบ lazy โดยค่าตั้งต้น** ฟังก์ชัน create จะยังไม่ทำงานจนกว่าจะมีวิดเจ็ตตัวแรกร้องขอ provider ตัวนั้นจริง ๆ พฤติกรรมนี้ช่วยประหยัดทรัพยากรตอนเปิดแอป แต่ถ้า provider ตัวใดต้องเริ่มทำงานทันทีตั้งแต่แอปเปิด เช่นต้องเชื่อมต่อกับบริการเบื้องหลัง ให้กำหนด lazy: false ให้ provider ตัวนั้น

11.8 เมื่อ Provider ตัวเดียวใหญ่เกินไป: การแบ่งความรับผิดชอบ

MultiProvider ตอบคำถามว่า "ประกาศ provider หลายตัวอย่างไรให้สะอาด" แต่ยังไม่ได้ตอบคำถามที่สำคัญกว่า นั่นคือ "ควรหั่น provider ออกเป็นกี่ตัว และหั่นตรงไหน" ซึ่งเป็นคำถามที่บทที่ 10 ทิ้งท้ายไว้

เกณฑ์ที่ใช้ได้จริงคือ หั่นตามขอบเขตความรับผิดชอบ ไม่ใช่หั่นตามหน้าจอ [11] provider หนึ่งตัวควรเป็นเจ้าของข้อมูลหนึ่งชุดที่เปลี่ยนแปลงไปด้วยกัน และมีเหตุผลเดียวที่จะต้องถูกแก้ไข ถ้าเราพบว่าตัวเองกำลังแก้ provider ตัวเดียวกันด้วยเหตุผลที่ไม่เกี่ยวข้องกันเลย เช่น แก้เพราะกติกาการคิดส่วนลดเปลี่ยน และแก้เพราะรูปแบบการแสดงประวัติการแชทเปลี่ยน นั่นคือสัญญาณว่าควรหั่นออกจากกัน

- **สัญญาณว่าควรหั่น** ไฟล์ provider ยาวจนต้องเลื่อนหาเมธอดนาน มีกลุ่มข้อมูลที่ไม่เคยเปลี่ยนพร้อมกันอยู่ในคลาสเดียว หรือการเรียก notifyListeners() หนึ่งครั้งไปปลุกวิดเจ็ตที่ไม่เกี่ยวข้องกันเป็นจำนวนมาก
- **สัญญาณว่าหั่นมากเกินไป** provider หลายตัวต้องถูกแก้พร้อมกันเสมอเมื่อมีฟีเจอร์ใหม่ หรือต้องคอยส่งข้อมูลไปมาระหว่างกันตลอดเวลาจนตามอ่านลำดับการทำงานไม่ทัน

แล้วถ้า provider ตัวหนึ่งจำเป็นต้องใช้ข้อมูลของอีกตัวหนึ่ง เช่น CartProvider ต้องรู้ว่าผู้ใช้คนใดกำลังล็อกอินอยู่ เพื่อดึงตะกร้าสินค้าของคนนั้น แพ็กเกจ provider เตรียม ChangeNotifierProxyProvider ไว้สำหรับกรณีนี้โดยเฉพาะ มันคือ provider ที่ประกาศได้ว่าตัวเองพึ่งพา provider ตัวใด และจะเรียกฟังก์ชัน update ให้ทุกครั้งที่ provider ต้นทางเปลี่ยนแปลง [12]

```
// lib/main.dart
MultiProvider(
  providers: [
    ChangeNotifierProvider(create: (context) => AuthProvider()),

    // CartProvider ต้องรู้ว่าใครล็อกอินอยู่ จึงต้องประกาศ 'หลัง' AuthProvider
    ChangeNotifierProxyProvider<AuthProvider, CartProvider>(
      create: (context) => CartProvider(),
      // update ถูกเรียกทุกครั้งที่ AuthProvider ประกาศการเปลี่ยนแปลง
      // ต้องนำ previousCart กลับมาใช้ซ้ำ ไม่เช่นนั้นข้อมูลในตะกร้าจะหายทุกครั้ง
      update: (context, auth, previousCart) =>
          (previousCart ?? CartProvider())..setUser(auth.userId),
    ),
  ],
  child: const MyApp(),
)
```

กับดักของ ChangeNotifierProxyProvider ฟังก์ชัน update ต้องคืนค่าเป็นอ็อบเจกต์เดิมที่ปรับค่าแล้ว ไม่ใช่สร้างอ็อบเจกต์ใหม่ทุกครั้ง เพราะการสร้างใหม่หมายถึงการทิ้ง state ที่สะสมไว้ทั้งหมด ผู้เรียนจำนวนมากพบอาการสินค้าในตะกร้าหายเองโดยไม่ทราบสาเหตุ ซึ่งมีต้นตอมาจากจุดนี้

นอกจากนี้ ถ้าพบว่าตัวเองต้องใช้ ChangeNotifierProxyProvider ซ้อนกันหลายชั้นจนตามลำดับการพึ่งพาไม่ทัน ให้ถอยกลับมาทบทวนการแบ่งความรับผิดชอบอีกครั้ง เพราะนั่นมักแปลว่าเส้นแบ่งระหว่าง provider ถูกลากผิดที่ตั้งแต่แรก

11.9 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

ในบทนี้ เราได้ศึกษาเครื่องมือขั้นสูงของ Provider ที่ช่วยให้แอปพลิเคชันของเรามีประสิทธิภาพและเป็นระเบียบมากขึ้น:

- **Consumer<T>:** เป็นเครื่องมือสำคัญในการเพิ่มประสิทธิภาพ โดยจะสร้างวิดเจ็ตใหม่เฉพาะส่วนของ UI ที่จำเป็นต้องอัปเดตตามสถานะเท่านั้น
- **MultiProvider:** ช่วยให้การประกาศ Provider หลายๆ ตัวในแอปพลิเคชันทำได้อย่างสะอาดตาและจัดการได้ง่าย
- **context.select<T, R>() และ Selector<T, R>:** ตีกรอบการรับฟังลงไปถึงระดับค่าข้อมูลรายตัว โดยเปรียบเทียบค่าที่เลือกด้วย == ก่อนตัดสินใจสร้าง

ใหม่ ข้อแม้คือค่าที่เลือกต้องเปรียบเทียบด้วย == ได้อย่างมีความหมาย

- **บันไดของการรับฟัง:** read ไม่รับฟังเลย, watch รับฟังทั้งเมธอด build, Consumer รับฟังเฉพาะพื้นที่ที่ห่อไว้, select รับฟังเฉพาะค่าที่ระบุ ให้เริ่มจากตัวที่อ่านง่ายที่สุดแล้วค่อยไต่ลงเมื่อวัดผลแล้วพบปัญหาจริง
- **การวัดผล:** ใช้ print ใน builder, debugPrintRebuildDirtyWidgets หรือ Flutter DevTools เพื่อพิสูจน์ว่าขอบเขตการ rebuild แคบลงจริง และต้องวัดในโหมด profile เท่านั้น
- **การแบ่งความรับผิดชอบของ Provider:** หั่นตามเหตุผลที่จะต้องถูกแก้ไข ไม่ใช่ตามหน้าจอ และใช้ ChangeNotifierProxyProvider เมื่อ provider ตัวหนึ่งจำเป็นต้องพึ่งพาข้อมูลของอีกตัวหนึ่ง

บัดนี้ คุณมีเครื่องมือที่ครบครันในการจัดการสถานะที่ซับซ้อนแล้ว ส่วนปฏิบัติการถัดไปจะให้คุณลงมือสร้างแอปพลิเคชันที่ใช้แนวคิดเหล่านี้อย่างเต็มรูปแบบ เพื่อเป็นรากฐานที่มั่นคงก่อนจะนำการเชื่อมต่อเครือข่ายเข้ามาในบทที่ 12 และฐานข้อมูลในบทที่ 13

11.10 คำถามชวนคิดท้ายบท

ตลอดทั้งบทนี้ ข้อมูลทุกชิ้นที่ provider ถือไว้ล้วนอยู่ในหน่วยความจำและหยิบใช้ได้ทันที เมื่อผู้ใช้กดบันทึก ค่าในลิสต์ก็เปลี่ยนในบรรทัดถัดไป แล้ว notifyListeners() ก็ประกาศผลออกไปได้เลย ทุกอย่างเสร็จสิ้นภายในเสี้ยววินาทีเดียวกัน

ความเร็วนั้นมีราคาที่ต้องจ่าย ข้อมูลทั้งหมดอยู่บนหน่วยความจำชั่วคราวภายในเครื่องเท่านั้น จึงเชื่อถือได้เต็มร้อยตราบที่แอปยังเปิดอยู่ แต่จะสูญสลายไปทันทีที่ผู้ใช้ปัดหน้าจอปิดแอปพลิเคชัน

แต่ในแอปพลิเคชันจริง รายการสินค้าไม่ได้ถูกเขียนไว้ในโค้ด มันต้องถูกดึงมาจากเซิร์ฟเวอร์ที่อาจตั้งอยู่อีกทวีปหนึ่ง ผ่านอินเทอร์เน็ตซึ่งอาจใช้เวลาครึ่งวินาที สามวินาที หรืออาจล้มเหลวกลางทางเพราะสัญญาณหลุดหรือเซิร์ฟเวอร์ล่ม โครงสร้างที่แยกส่วนอย่างสวยงามและลื่นไหลที่เราเพิ่งจัดระเบียบไว้ กำลังจะต้องเผชิญหน้ากับความไม่แน่นอนของโลกภายนอก

คำถามคือ ระหว่างที่ข้อมูลยังเดินทางมาไม่ถึง provider ควรบอกหน้าจอว่าอย่างไร ถ้าตอบว่า "ลิสต์ว่าง" ผู้ใช้จะเห็นหน้าจอเปล่าและเข้าใจผิดว่าไม่มีสินค้าเลย และถ้าการเชื่อมต่อล้มเหลว ใครควรเป็นผู้จำข้อผิดพลาดนั้นไว้เพื่อแสดงปุ่มลองใหม่

ลองคิดว่า ItemProvider ของเราจะต้องเก็บอะไรเพิ่มอีกบ้างนอกจากตัวข้อมูล และการเพิ่มสิ่งเหล่านั้นจะเปลี่ยนหน้าตาของ Consumer ในหน้าจอหลักไปอย่างไร แล้วนำคำตอบของตัวเองไปเทียบกับบทที่ 12: การเขียนโปรแกรมแบบ Asynchronous และ Networking ซึ่งจะแนะนำ Future, async, await และการจัดการสถานะกำลังโหลดกับสถานะผิดพลาดอย่างเป็นระบบ

เอกสารอ้างอิง (References)

[1] Flutter Team, "ChangeNotifier class," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/foundation/ChangeNotifier-class.html

[2] Flutter Team, "State management introduction," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/data-and-backend/state-mgmt/simple

[3] Flutter Team, "Consumer class," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/documentation/provider/latest/provider/Consumer-class.html

[4] Flutter Team, "provider package," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/packages/provider

[5] Flutter Team, "Selector class," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/documentation/provider/latest/provider/Selector-class.html

[6] Flutter Team, "debugPrintRebuildDirtyWidgets property," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/cupertino/debugPrintRebuildDirtyWidgets.html

[7] Flutter Team, "Android Studio and IntelliJ," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/tools/android-studio

[8] Flutter Team, "Use the Performance view," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/tools/devtools/performance

[9] Flutter Team, "Flutter performance profiling," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/perf/ui-performance

[10] Flutter Team, "MultiProvider class," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/documentation/provider/latest/provider/MultiProvider-class.html

[11] Flutter Team, "Common architecture concepts," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/app-architecture/concepts

[12] Flutter Team, "ChangeNotifierProxyProvider class," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/documentation/provider/latest/provider/ChangeNotifierProxyProvider-class.html

ปฏิบัติการที่ 1 บทที่ 11: สร้างแอปพลิเคชันแก้ไขรายการด้วย Provider และ Consumer

วัตถุประสงค์: เพื่อฝึกฝนการใช้ ChangeNotifierProvider, Consumer และการนำทางเพื่อสร้างแอปพลิเคชันสองหน้าจอที่สามารถแชร์และแก้ไขสถานะได้ โดยยังไม่ต้องใช้ฐานข้อมูล

เวลาโดยประมาณ: 2 ชั่วโมง

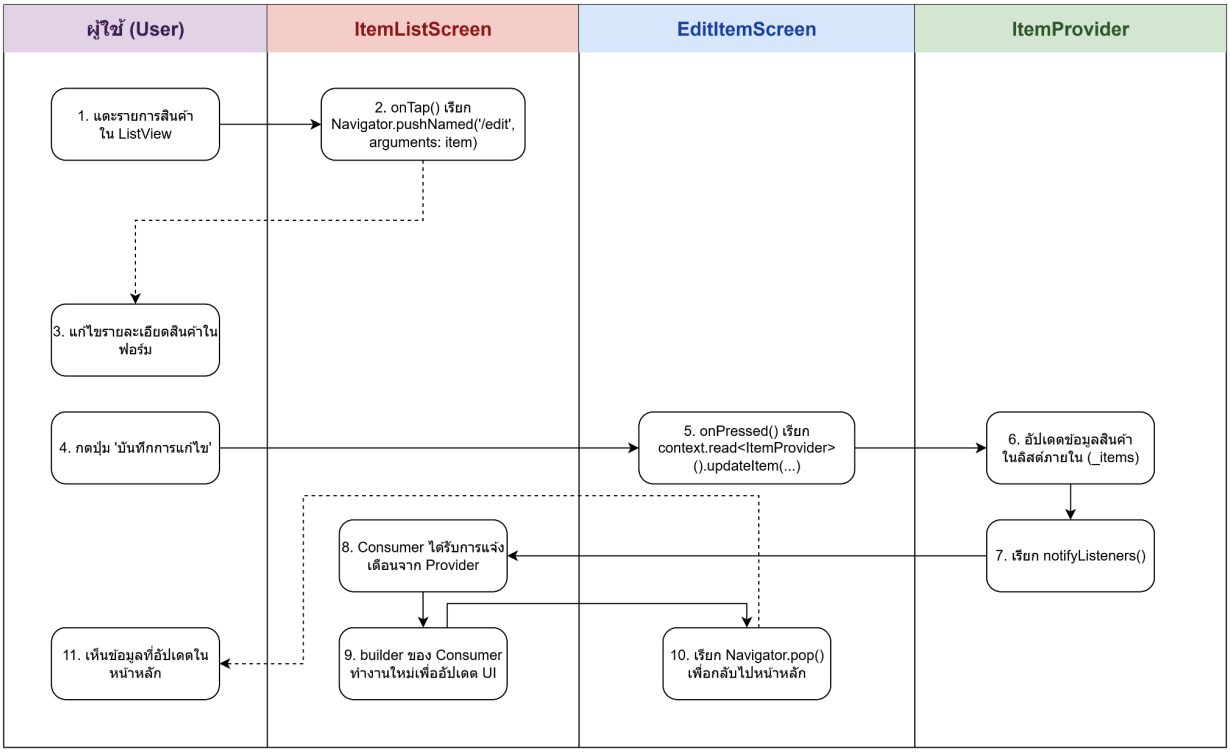
ขั้นตอนที่ 1: การตั้งค่าโปรเจกต์และ Dependencies

1. **คำแนะนำ:** สร้างโปรเจกต์ Flutter ใหม่ชื่อ provider_editor_lab.
2. **คำแนะนำ:** เปิดไฟล์ pubspec.yaml และเพิ่ม provider ในส่วน dependencies.

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.2 # ตรวจสอบเวอร์ชันล่าสุดบน pub.dev
  cupertino_icons: ^1.0.2
```

3. **คำแนะนำ:** สั่ง flutter pub get ในเครื่องมือ Terminal.
4. **คำแนะนำ:** ใน lib, สร้างโฟลเดอร์ models, providers, และ screens เพื่อจัดระเบียบไฟล์.
5. **คำแนะนำ:** ลำดับการทำงานของโปรเจคนี้จะเป็นดังแผนผังด้านล่าง

แผนภาพกระบวนการ: ปฏิบัติการบทที่ 11



ขั้นตอนที่ 2: การสร้างคลาสต้นแบบข้อมูล (Data Model)

1. **คำแนะนำ:** ในโฟลเดอร์ lib/models, สร้างไฟล์ item.dart.
2. **คำแนะนำ:** สร้างคลาส Item เพื่อเป็นต้นแบบของข้อมูล ซึ่งประกอบด้วย id, name, และ description.

```dart
// lib/models/item.dart
class Item {
  final String id;
  String name;
  String description;

  Item({required this.id, required this.name, required this.description});
}
```

ขั้นตอนที่ 3: การสร้าง ItemProvider

1. **คำแนะนำ:** ในโฟลเดอร์ lib/providers, สร้างไฟล์ item_provider.dart.

2. **คำแนะนำ:** สร้างคลาส ItemProvider ที่สืบทอดคุณสมบัติจาก ChangeNotifier และ implement ตรรกะการจัดการข้อมูล พร้อมข้อมูลตัวอย่างเริ่มต้น. คลาสนี้จะทำหน้าที่เป็น "แหล่งข้อมูลจริงเพียงแหล่งเดียว" (Single Source of Truth) สำหรับข้อมูลรายการสินค้าของเรา.

```dart
// lib/providers/item_provider.dart
import 'package:flutter/foundation.dart';
import '../models/item.dart';

class ItemProvider extends ChangeNotifier {
  // ข้อมูลตัวอย่างเริ่มต้น
  final List<Item> _items = [
    Item(id: '1', name: 'MacBook Pro', description: 'Apple M3 Pro Chip'),
    Item(id: '2', name: 'Dell XPS 15', description: 'Intel Core i9, 32GB RAM'),
    Item(id: '3', name: 'Logitech MX Master 3S', description: 'Wireless Mouse'),
  ];

  // Getter เพื่อให้ส่วน UI สามารถเข้าถึงข้อมูลได้
  List<Item> get items => _items;

  // เมธอดสำหรับอัปเดตข้อมูลตาม id
  void updateItem(String id, String newName, String newDescription) {
    final itemIndex = _items.indexWhere((item) => item.id == id);
    if (itemIndex >= 0) {
      _items[itemIndex].name = newName;
      _items[itemIndex].description = newDescription;
      notifyListeners(); // แจ้งเตือนวิดเจ็ตที่กำลัง "ฟัง" อยู่ว่ามีการเปลี่ยนแปลง
    }
  }
}
```

ขั้นตอนที่ 4: การสร้างส่วนติดต่อผู้ใช้ (UI)

1. **คำแนะนำ:** ในโฟลเดอร์ lib/screens, สร้างไฟล์ item_list_screen.dart (หน้าหลัก). เราจะใช้ Consumer เพื่อแสดง ListView อย่างมีประสิทธิภาพ.

```dart
// lib/screens/item_list_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/item_provider.dart';
import 'edit_item_screen.dart';

class ItemListScreen extends StatelessWidget {
  static const String routeName = '/';
  const ItemListScreen({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('รายการสินค้า')),
      body: Consumer<ItemProvider>(
        builder: (context, itemProvider, child) {
          return ListView.builder(
            itemCount: itemProvider.items.length,
            itemBuilder: (context, index) {
              final item = itemProvider.items[index];
              return ListTile(
                title: Text(item.name),
                subtitle: Text(item.description),
                trailing: const Icon(Icons.edit),
                onTap: () {
                  Navigator.pushNamed(
                    context,
                    EditItemScreen.routeName,
                    arguments: item, // ส่งอ็อบเจกต์ Item ทั้งหมดไปกับ route
                  );
                },
              );
            },
          );
        },
      ),
    );
  }
}
```

สังเกตขอบเขตของ Consumer ในโค้ดนี้: เราห่อ ListView.builder ทั้งก้อนไว้ใน Consumer ซึ่งหมายความว่าเมื่อสินค้ารายการใดรายการหนึ่งถูกแก้ไข รายการทั้งหมดบนหน้าจอจะถูกสร้างใหม่ สำหรับแล็บนี้ที่มีข้อมูลเพียงสามรายการ ต้นทุนดังกล่าวต่ำมากจนไม่ต้องกังวล และการเขียนแบบนี้อ่านง่ายที่สุด

แต่ถ้ารายการมีหลายร้อยแถวและแต่ละแถวมีรูปภาพ การตีกรอบที่เหมาะสมกว่าคือย้าย Consumer หรือ Selector ลงไปไว้ ภายใน itemBuilder เพื่อให้แต่ละแถวรับฟังเฉพาะข้อมูลของตัวเอง นี่คือรูปธรรมของบันไดการรับฟังในหัวข้อ 11.4 ที่ว่าให้เลือกระดับตามขนาดของปัญหาจริง ไม่ใช่ตีกรอบให้แคบที่สุดไว้ก่อนเสมอ

2. **คำแนะนำ:** ในโฟลเดอร์ lib/screens, สร้างไฟล์ edit_item_screen.dart (หน้ารายละเอียด).

```
// lib/screens/edit_item_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/item.dart';
import '../providers/item_provider.dart';
```

```dart
class EditItemScreen extends StatefulWidget {
  static const String routeName = '/edit-item';
  const EditItemScreen({super.key});

  @override
  State<EditItemScreen> createState() => _EditItemScreenState();
}

class _EditItemScreenState extends State<EditItemScreen> {
  late final TextEditingController _nameController;
  late final TextEditingController _descriptionController;
  late Item _item;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    // ดึงข้อมูล Item ที่ถูกส่งมาจากหน้า ListScreen
    // เราใช้ didChangeDependencies เพราะ ModalRoute.of(context) ไม่ควรถูกเรียกใน
initState
    final item = ModalRoute.of(context)!.settings.arguments as Item;
    _item = item;

    // กำหนดค่าเริ่มต้นให้กับ Controller จากข้อมูลที่ได้รับ
    _nameController = TextEditingController(text: _item.name);
    _descriptionController = TextEditingController(text: _item.description);
  }

  @override
  void dispose() {
    _nameController.dispose();
    _descriptionController.dispose();
    super.dispose();
  }

  void _saveForm() {
    // ใช้ context.read เพื่อเรียกเมธอดใน provider โดยไม่ทำให้หน้านี้ rebuild
    context.read<ItemProvider>().updateItem(
          _item.id,
          _nameController.text,
          _descriptionController.text,
        );
    Navigator.pop(context); // กลับไปยังหน้าหลัก
  }

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: Text('แก้ไข: ${_item.name}')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            TextFormField(
              controller: _nameController,
              decoration: const InputDecoration(labelText: 'ชื่อสินค้า'),
            ),
            TextFormField(
              controller: _descriptionController,
              decoration: const InputDecoration(labelText: 'คำอธิบาย'),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: _saveForm,
              child: const Text('บันทึกการเปลี่ยนแปลง'),
            )
          ],
        ),
      ),
    );
  }
}
```

ทำไมโค้ดนี้จึงดึงข้อมูลใน didChangeDependencies ไม่ใช่ initState: โดยทั่วไปเรามักกำหนดค่าเริ่มต้นของตัวแปรไว้ใน initState ซึ่งเป็นจังหวะแรกสุดของวงจรชีวิตหน้าจอ แต่ในกรณีนี้กลับทำไม่ได้ และเป็นจุดที่นักพัฒนาจำนวนมากเจ็บตัวมาแล้ว

เหตุผลเชิงสถาปัตยกรรมคือ ในจังหวะ initState ตัวแปร context ยังเชื่อมต่อกับ widget tree ด้านบนไม่สมบูรณ์ เปรียบเสมือนปลั๊กไฟที่เพิ่งเสียบเข้าเต้ารับแต่ยังไม่ได้เปิดสวิตช์ วิดเจ็ตรู้ว่าตัวเองมีตัวตนแล้ว แต่ยังไม่รู้ว่าตัวเองอยู่ตรงไหนของโครงสร้าง

ดังนั้นถ้าเราเรียก ModalRoute.of(context) ในจังหวะนั้นเพื่อวิ่งขึ้นไปหาข้อมูล route ระบบจะหยุดทำงานทันทีเพราะหาทางเดินขึ้นไปไม่เจอ เราจึงต้องย้ายคำสั่งมาไว้ที่ didChangeDependencies ซึ่งเป็นวงจรชีวิตลำดับถัดมา เป็นจังหวะที่ไฟเดินครบวงจรแล้ว คือ context รับรู้โครงสร้างทั้งหมดและวิ่งไปสกัด argument จาก route ออกมาได้อย่างปลอดภัย

ข้อควรระวังสำคัญของโค้ดหน้าแก้ไข: เมธอด didChangeDependencies ไม่ได้ถูกเรียกเพียงครั้งเดียว Flutter จะเรียกมันซ้ำทุกครั้งที่สิ่งที่วิดเจ็ตนี้พึ่งพาเปลี่ยนแปลง เช่น เมื่อธีมของแอปเปลี่ยน เมื่อขนาดหน้าจอเปลี่ยนจากการหมุนเครื่อง หรือเมื่อมีการเปลี่ยนภาษา

เมื่อถูกเรียกครั้งที่สอง โค้ดข้างต้นจะพยายามกำหนดค่าให้ตัวแปรที่ประกาศเป็น late final ซ้ำอีกรอบ ผลคือโปรแกรมจะหยุดทำงานพร้อมข้อความ LateInitializationError และยังทำให้ TextEditingController ตัวเดิมค้างอยู่ในหน่วยความจำโดยไม่มีใคร dispose ให้ด้วย

แนวทางแก้ที่นิยมใช้คือวาง "ยาม" ไว้ให้โค้ดส่วนนี้ทำงานเพียงครั้งแรกครั้งเดียว ดังตัวอย่างด้านล่าง ให้นำไปใช้แทนเมธอด didChangeDependencies เดิม โค้ดของปฏิบัติการที่ 2 ในบทนี้ใช้รูปแบบยามแบบเดียวกัน ลองเปิดเทียบกันดูเพื่อให้เห็นว่าเป็นแพตเทิร์นมาตรฐาน ไม่ใช่กลเม็ดเฉพาะกิจ

```dart
class _EditItemScreenState extends State<EditItemScreen> {
  late final TextEditingController _nameController;
  late final TextEditingController _descriptionController;
  late Item _item;
  bool _isInit = false; // ยามกันการกำหนดค่าซ้ำ

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    if (_isInit) return; // ครั้งที่สองเป็นต้นไป ไม่ต้องสร้าง controller ใหม่
    _isInit = true;

    _item = ModalRoute.of(context)!.settings.arguments as Item;
    _nameController = TextEditingController(text: _item.name);
    _descriptionController = TextEditingController(text: _item.description);
  }

  // เมธอด dispose, _saveForm และ build คงเดิมทุกประการ
}
```

ขั้นตอนที่ 5: การเชื่อมต่อทั้งหมดใน main.dart

1. **คำแนะนำ:** แก้ไข lib/main.dart เพื่อตั้งค่า ChangeNotifierProvider ที่ระดับบนสุดของแอป และกำหนด routes สำหรับการนำทาง.

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/item_provider.dart';
import 'screens/item_list_screen.dart';
import 'screens/edit_item_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => ItemProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      title: 'Provider Editor Lab',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.indigo),
      initialRoute: ItemListScreen.routeName,
      routes: {
        ItemListScreen.routeName: (context) => const ItemListScreen(),
        EditItemScreen.routeName: (context) => const EditItemScreen(),
      },
    );
  }
}
```

ขั้นตอนที่ 6: การรันและทดสอบ

1. **คำแนะนำ:** รันแอปพลิเคชันด้วยการรีสตาร์ทเต็มรูปแบบ (Full Restart).
2. **คำแนะนำ:** คุณจะเห็น "หน้าหลัก" พร้อมรายการสินค้า 3 อย่าง.
3. **คำแนะนำ:** แตะที่รายการ "Dell XPS 15". คุณจะถูกนำไปยังหน้า "แก้ไขรายการ".
4. **คำแนะนำ:** แก้ไข "คำอธิบาย" เป็น "Intel Core Ultra 7, 64GB RAM" แล้วกด "บันทึกการเปลี่ยนแปลง".
5. **คำแนะนำ:** คุณจะกลับมาที่ "หน้าหลัก". สังเกตว่าคำอธิบายของ "Dell XPS 15" ได้รับการอัปเดตเรียบร้อยแล้ว. การเปลี่ยนแปลงนี้เกิดขึ้นเนื่องจาก Consumer ใน ItemListScreen ได้รับการแจ้งเตือนจาก ItemProvider และทำการสร้าง ListView ใหม่อีกครั้ง.

ยินดีด้วย! คุณได้สร้างแอปที่จัดการสถานะข้ามหน้าจอโดยใช้ Provider และ Consumer สำเร็จแล้ว. นี่คือรากฐานที่แข็งแกร่งสำหรับการสร้างแอปที่ซับซ้อนและต่อยอดไปสู่การใช้ฐานข้อมูลในบทถัดไป.

ทดลองต่อยอดเพื่อพิสูจน์ด้วยตาตัวเอง: ให้เพิ่มคำสั่ง print ไว้สองจุด จุดแรกอยู่บรรทัดแรกของเมธอด build ใน ItemListScreen และจุดที่สองอยู่บรรทัดแรกของ builder ใน Consumer แล้วลองแก้ไขข้อมูลสินค้าสักสามครั้ง

ผลที่ควรเห็นใน console คือข้อความจากเมธอด build ปรากฏเพียงครั้งเดียวตอนเปิดหน้าจอ ขณะที่ข้อความจาก builder ปรากฏเพิ่มทุกครั้งที่บันทึกการแก้ไข นี่คือหลักฐานว่า Consumer ตีกรอบการสร้างใหม่ไว้เฉพาะภายในขอบเขตของมันจริง ตามที่อธิบายไว้ในหัวข้อ 11.5

กิจกรรมปฏิบัติการที่ 2 บทที่ 11: สร้างแอปจัดการรายการสินค้า

วัตถุประสงค์: เพื่อฝึกฝนการใช้ ChangeNotifierProvider, Consumer, และ MultiProvider ในการสร้างแอปพลิเคชันที่มีหลายหน้าจอและมีการจัดการสถานะร่วมกัน.

เวลาโดยประมาณ: 2 ชั่วโมง

ขั้นตอนที่ 1: การตั้งค่าโปรเจกต์และ Dependencies

1. **คำแนะนำ:** สร้างโปรเจกต์ Flutter ใหม่ชื่อ provider_shop
2. **คำแนะนำ:** เปิดไฟล์ pubspec.yaml และเพิ่ม provider ใน dependencies

   dependencies:

   flutter:

   sdk: flutter

   provider: ^6.1.5+1 # ตรวจสอบเวอร์ชันล่าสุดบน pub.dev

   cupertino_icons: ^1.0.2

3. **คำแนะนำ:** รัน flutter pub get ใน terminal เพื่อติดตั้งแพ็กเกจ.

ขั้นตอนที่ 2: การสร้าง Provider

1. **คำแนะนำ:** สร้างโฟลเดอร์ใหม่ใน lib ชื่อ models และสร้างไฟล์ item.dart ภายในนั้น

```dart
// lib/models/item.dart
class Item {
  final String id;
  String name;
  String description;
  double price;

  Item({
    required this.id,
    required this.name,
    required this.description,
```

```
    required this.price,
  });
}
```

2. **คำแนะนำ:** สร้างโฟลเดอร์ใหม่ใน lib ชื่อ providers และสร้างไฟล์ 2 ไฟล์:
   - item_provider.dart
   - user_profile_provider.dart
3. **คำแนะนำ:** เพิ่มโค้ดสำหรับ ItemProvider ใน item_provider.dart

```dart
// lib/providers/item_provider.dart
import 'package:flutter/foundation.dart';
import '../models/item.dart';

class ItemProvider extends ChangeNotifier {
  final List<Item> _items = [
    Item(id: 'i1', name: 'ปากกาสีน้ำเงิน', description: 'หมึกเจล เขียนลื่น', price:
25.0),
    Item(id: 'i2', name: 'สมุดบันทึกปกแข็ง', description: 'ขนาด A5, 120 แกรม', price:
89.0),
    Item(id: 'i3', name: 'คลิปหนีบกระดาษ', description: 'กล่องละ 50 ชิ้น, สีเงิน',
price: 30.0),
  ];

  List<Item> get items => [..._items]; // คืนค่าสำเนาของลิสต์

  Item findById(String id) {
    return _items.firstWhere((item) => item.id == id);
  }

  void updateItem(String id, Item newItemData) {
    final itemIndex = _items.indexWhere((item) => item.id == id);
    if (itemIndex >= 0) {
      _items[itemIndex] = newItemData;
      notifyListeners(); // แจ้งเตือนการเปลี่ยนแปลง
    }
  }
}
```

4. **คำแนะนำ:** เพิ่มโค้ดสำหรับ UserProfileProvider ใน user_profile_provider.dart

```dart
// lib/providers/user_profile_provider.dart
import 'package:flutter/foundation.dart';

class UserProfileProvider extends ChangeNotifier {
  String _username = 'Student101'; // ชื่อผู้ใช้เริ่มต้น

  String get username => _username;

  void changeUsername(String newName) {
    _username = newName;
    notifyListeners();
  }
}
```

ขั้นตอนที่ 3: การสร้างหน้าจอ (Screens)

1. **คำแนะนำ:** สร้างโฟลเดอร์ใหม่ใน lib ชื่อ screens และสร้างไฟล์ 2 ไฟล์:
   - item_list_screen.dart (หน้าหลัก)
   - edit_item_screen.dart (หน้าแก้ไข)
2. **คำแนะนำ:** เพิ่มโค้ดสำหรับ ItemListScreen ซึ่งจะแสดงรายการสินค้า

```dart
// lib/screens/item_list_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/item_provider.dart';
import '../providers/user_profile_provider.dart'; // import provider ผู้ใช้
import 'edit_item_screen.dart';

class ItemListScreen extends StatelessWidget {
  static const routeName = '/';
  const ItemListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // ดึงชื่อผู้ใช้มาแสดงใน AppBar
    final username = context.watch<UserProfileProvider>().username;

    return Scaffold(
      appBar: AppBar(
        title: Text('รายการสินค้า (ผู้ใช้: $username)'),
      ),
      body: Consumer<ItemProvider>(
        builder: (ctx, itemProvider, child) => ListView.builder(
```

```
          itemCount: itemProvider.items.length,
          itemBuilder: (_, i) {
            final item = itemProvider.items[i];
            return ListTile(
              title: Text(item.name),
              subtitle: Text(item.description),
              trailing: Text('฿${item.price.toStringAsFixed(2)}'),
              onTap: () {
                Navigator.of(context).pushNamed(
                  EditItemScreen.routeName,
                  arguments: item.id,
                );
              },
            );
          },
        ),
      ),
    );
  }
}
```

3. **คำแนะนำ:** เพิ่มโค้ดสำหรับ EditItemScreen ซึ่งจะเป็นฟอร์มสำหรับแก้ไข

```
// lib/screens/edit_item_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/item.dart';
import '../providers/item_provider.dart';

class EditItemScreen extends StatefulWidget {
  static const routeName = '/edit-item';
  const EditItemScreen({super.key});

  @override
  State<EditItemScreen> createState() => _EditItemScreenState();
}

class _EditItemScreenState extends State<EditItemScreen> {
  final _formKey = GlobalKey<FormState>();
  late String _itemId;
  late Item _editedItem;
  bool _isInit = false;

  // ใช้ TextEditingController เพื่อควบคุมค่าใน TextFormField
```

```
  final TextEditingController _nameController = TextEditingController();
  final TextEditingController _descController = TextEditingController();
  final TextEditingController _priceController = TextEditingController();

  @override
  void didChangeDependencies() {
    if (!_isInit) {
      _itemId = ModalRoute.of(context)!.settings.arguments as String;
      _editedItem = Provider.of<ItemProvider>(context, listen:
false).findById(_itemId);

      // ตั้งค่าเริ่มต้นให้ controller
      _nameController.text = _editedItem.name;
      _descController.text = _editedItem.description;
      _priceController.text = _editedItem.price.toString();
    }
    _isInit = true;
    super.didChangeDependencies();
  }

  @override
  void dispose() {
    // คืนทรัพยากร controller เมื่อวิดเจ็ตถูกทำลาย
    _nameController.dispose();
    _descController.dispose();
    _priceController.dispose();
    super.dispose();
  }

  void _saveForm() {
    final isValid = _formKey.currentState!.validate();
    if (!isValid) {
      return;
    }
    _formKey.currentState!.save();

    // ใช้ context.read เพื่อเรียกเมธอดจาก Provider
    context.read<ItemProvider>().updateItem(_itemId, _editedItem);
    Navigator.of(context).pop();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('แก้ไขรายละเอียดสินค้า'),
        actions: [
```

```
        IconButton(
          icon: const Icon(Icons.save),
          onPressed: _saveForm,
        ),
      ],
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: ListView(
          children: <Widget>[
            TextFormField(
              controller: _nameController,
              decoration: const InputDecoration(labelText: 'ชื่อสินค้า'),
              textInputAction: TextInputAction.next,
              validator: (value) {
                if (value!.isEmpty) return 'กรุณาป้อนชื่อสินค้า';
                return null;
              },
              onSaved: (value) {
                _editedItem = Item(
                    id: _editedItem.id,
                    name: value!,
                    description: _editedItem.description,
                    price: _editedItem.price);
              },
            ),
            TextFormField(
              controller: _descController,
              decoration: const InputDecoration(labelText: 'รายละเอียด'),
              maxLines: 3,
              keyboardType: TextInputType.multiline,
              validator: (value) {
                if (value!.isEmpty) return 'กรุณาป้อนรายละเอียด';
                return null;
              },
              onSaved: (value) {
                _editedItem = Item(
                    id: _editedItem.id,
                    name: _editedItem.name,
                    description: value!,
                    price: _editedItem.price);
              },
            ),
            TextFormField(
              controller: _priceController,
```

```
                decoration: const InputDecoration(labelText: 'ราคา'),
                textInputAction: TextInputAction.done,
                keyboardType: TextInputType.number,
                validator: (value) {
                  if (value!.isEmpty) return 'กรุณาป้อนราคา';
                  if (double.tryParse(value) == null) return 'กรุณาป้อนตัวเลขที่ถูกต้อง
';
                  if (double.parse(value) <= 0) return 'ราคาต้องมากกว่า 0';
                  return null;
                },
                onSaved: (value) {
                  _editedItem = Item(
                      id: _editedItem.id,
                      name: _editedItem.name,
                      description: _editedItem.description,
                      price: double.parse(value!));
                },
                onFieldSubmitted: (_) => _saveForm(),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

ขั้นตอนที่ 4: การเชื่อมต่อทุกอย่างเข้าด้วยกันใน main.dart

1. **คำแนะนำ:** เปิด lib/main.dart และแทนที่โค้ดทั้งหมดด้วยโค้ดด้านล่าง ซึ่งจะใช้ MultiProvider และกำหนด routes สำหรับการนำทาง

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import './providers/item_provider.dart';
import './providers/user_profile_provider.dart';
import './screens/item_list_screen.dart';
import './screens/edit_item_screen.dart';

void main() {
  runApp(const MyApp());
```

```
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (ctx) => ItemProvider()),
        ChangeNotifierProvider(create: (ctx) => UserProfileProvider()),
      ],
      child: MaterialApp(
        title: 'Provider Shop',
        theme: ThemeData(
          primarySwatch: Colors.purple,
          colorScheme: ColorScheme.fromSwatch(primarySwatch: Colors.purple)
              .copyWith(secondary: Colors.deepOrange),
        ),
        initialRoute: '/',
        routes: {
          ItemListScreen.routeName: (ctx) => const ItemListScreen(),
          EditItemScreen.routeName: (ctx) => const EditItemScreen(),
        },
      ),
    );
  }
}
```

ขั้นตอนที่ 5: ความท้าทายเพิ่มเติม

เมื่อแอปทำงานได้ครบถ้วนแล้ว ให้ทดลองต่อยอดสามข้อต่อไปนี้ เพื่อนำเครื่องมือในหัวข้อ 11.3 ถึง 11.8 มาใช้กับโค้ดที่เพิ่งเขียนเอง

- **เปลี่ยนจุดที่แสดงชื่อผู้ใช้บน AppBar ให้ใช้ context.select** โดยเลือกเฉพาะฟิลด์ชื่อจาก UserProfileProvider แล้วเปิด debugPrintRebuildDirtyWidgets เพื่อเทียบจำนวนครั้งที่วิดเจ็ตถูกสร้างใหม่ก่อนและหลังการแก้
- **ย้าย Consumer ลงไปไว้ใน itemBuilder** ให้แต่ละแถวของรายการรับฟังข้อมูลของตัวเอง แล้วสังเกตว่าเมื่อแก้ไขสินค้าเพียงรายการเดียว มีกี่แถวที่ถูกสร้างใหม่
- **ทดลองสลับลำดับ provider ใน MultiProvider** แล้วอธิบายว่าเหตุใดลำดับจึงสำคัญเมื่อ provider ตัวหนึ่งต้องพึ่งพาข้อมูลของอีกตัวหนึ่ง